

Ministry of Higher Education and Scientific Research

École Nationale Polytechnique, Algiers



Department of Electrical Engineering

Artificial intelligence

Laboratory report no. 1

**Implementation of a multilayer perceptron
trained by gradient backpropagation**

Submitted by:

LAGHA Khalil Ellah

MATALLAH Mohammed Achref

Set by:

Prof. A. HELLAL

Academic year 2023–2024

Abstract

This laboratory work covers the complete implementation of a multilayer perceptron in MATLAB without using any toolbox: forward propagation, gradient backpropagation and the training loop are written by hand in matrix form. The program is designed to be general, that is, able to handle any data set without modification of its computational core, since the network dimensions, the initial weights, the learning rate and the tolerance are all carried by the data files.

The program is validated on four data sets: three analytical single-observation examples and a small training set of three observations presented sequentially. The first three converge in 3 709, 528 537 and 5 835 iterations respectively, and the training set in 6 799, 6 875 and 6 875 iterations per observation, for a total time below one second.

The analysis of these results forms the main part of the report, because the differences between the examples are all explained by fundamental phenomena of neural network training. Example 2 needs a hundred times more iterations than the others for three cumulative reasons: disabled biases, a symmetric initialisation that prevents the two hidden neurons from ever differentiating, and a target set at 0.5, which the sigmoid reaches only asymptotically. On the training set the hidden-layer weights do not move at all: with raw inputs of the order of 230, the pre-activations reach several tens, the sigmoid saturates and its derivative becomes numerically zero, so the gradient no longer passes through the hidden layer. These two observations lead directly to the standard good practices of machine learning: random initialisation, retention of the biases and normalisation of the inputs.

Keywords: multilayer perceptron, gradient backpropagation, sigmoid activation, gradient descent, saturation, symmetry breaking, input normalisation, MATLAB.

Contents

Nomenclature	iv
1 Introduction	1
1.1 Context	1
1.2 Assignment	1
1.3 Outline	1
2 Theoretical background	1
2.1 Network architecture	1
2.2 Forward propagation	2
2.3 Gradient backpropagation	3
2.4 Stopping criterion	3
3 Program architecture	3
3.1 File organisation	3
3.2 Operation	4
3.3 The two implementations	5
4 Test data	5
5 Results	6
5.1 Examples 1 to 3	6
5.2 Training set	7
6 Discussion	8
6.1 Convergence speed of examples 1 and 3	8
6.2 The pathological case of example 2	8
6.3 Sigmoid saturation on the training set	9
6.4 Value of the vectorised version	9
7 Conclusion	9
A Complete data sets	11
B Reproducing the results	11

List of Figures

1	Architecture of the implemented multilayer perceptron: N inputs, K hidden neurons, S outputs, with one bias input per layer.	2
2	Sigmoid function and its derivative. Beyond $ u \approx 6$ the derivative is numerically zero: no gradient can flow back through a neuron in that state.	2
3	Convergence of the maximum output error for the three examples, logarithmic on both axes. Example 2 shows a long plateau before the error really begins to decrease.	7
4	Training set: error during the three chained training phases. The vertical lines mark the transition from one observation to the next.	8

List of Tables

2	Organisation of the program.	4
3	Characteristics of the four data sets.	6
4	Convergence of examples 1 to 3, loop version.	6
5	Training set: iterations per observation. Total time 0.99 s	7
6	Initial values of the four data sets.	11

Nomenclature

N, K, S	Numbers of input, hidden and output neurons	–
x, x_0	Input vector and hidden-layer bias input	–
z_0	Output-layer bias input	–
w_1, w_{01}	Hidden-layer weights and bias weights	–
w_2, w_{02}	Output-layer weights and bias weights	–
U_{in}, U_{out}	Hidden-layer pre-activations and outputs	–
Y_{in}, y	Network pre-activations and outputs	–
y_d	Desired output	–
e	Output error, $e = y_d - y$	–
f	Sigmoid activation function	–
δ_1, δ_2	Local gradients of the hidden and output layers	–
α	Learning rate	–
tol	Stopping-criterion tolerance	–
E	Quadratic error	–

1 Introduction

1.1 Context

The multilayer perceptron is one of the foundational models of machine learning. A neural network with a single hidden layer, trained by gradient backpropagation, can approximate any nonlinear function to arbitrary accuracy, a result known as the universal approximation theorem. This property makes it the basic tool for classification and regression problems, and the starting point of every deeper architecture developed since.

Its implementation rests on a single algorithm, backpropagation, whose idea is to answer the following question: every weight of the network contributes to the error observed at the output, but in what proportion? Backpropagation answers by applying the chain rule from the output back to the input, which computes the gradient of the error with respect to all weights in a single backward pass.

1.2 Assignment

The assignment asks for a MATLAB program implementing a multilayer perceptron applicable to any system or data set. The data for three different examples are provided, together with a small training set of three observations.

The central requirement is **generality**: the program must solve every example without modification of its code, using a matrix formulation. This constraint shaped the whole architecture described in Section 3: the network dimensions, the initial weights, the biases, the learning rate, the tolerance and the desired output are entirely carried by the data files, and the computational core knows nothing about them beyond their dimensions.

1.3 Outline

Section 2 reviews the theory of the multilayer perceptron and derives the backpropagation equations. Section 3 describes the program architecture and its two implementations. Section 4 presents the four test data sets. Section 5 collects the results and Section 6 analyses them, with particular attention to the two pathological cases that proved the most instructive. Section 7 concludes.

2 Theoretical background

2.1 Network architecture

The network has three layers, shown in Figure 1:

- an input layer of N variables $x = (x_1, \dots, x_N)^\top$, together with a bias input x_0 ;
- a hidden layer of K neurons, with weights $w_1 \in \mathbb{R}^{K \times N}$ and bias weights $w_{01} \in \mathbb{R}^K$;
- an output layer of S neurons, with weights $w_2 \in \mathbb{R}^{S \times K}$ and bias weights $w_{02} \in \mathbb{R}^S$.

Every neuron uses the sigmoid activation function

$$f(u) = \frac{1}{1 + e^{-u}}, \quad f'(u) = f(u) (1 - f(u)). \quad (1)$$

The form of the derivative is worth emphasising: it is expressed directly from the neuron's output, with no need to retain the pre-activation. This makes the gradient computation very

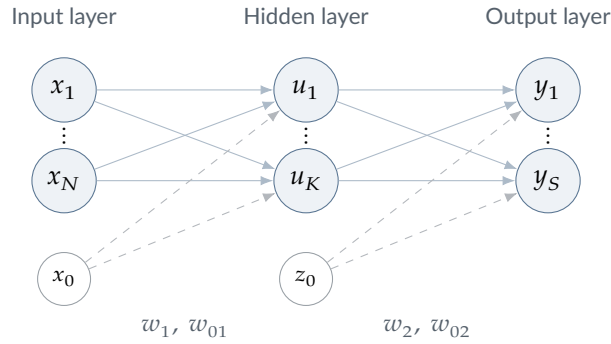


Figure 1. Architecture of the implemented multilayer perceptron: N inputs, K hidden neurons, S outputs, with one bias input per layer.

cheap, and it is historically one of the reasons for the popularity of the sigmoid. Figure 2 shows the function and its derivative, and makes visible the property that will explain the behaviour observed in Section 6.3: the derivative vanishes as soon as $|u|$ exceeds a few units.

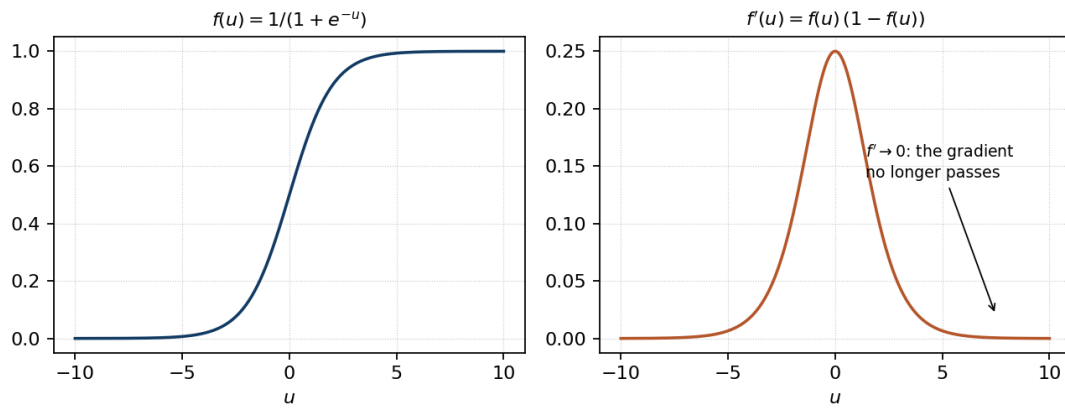


Figure 2. Sigmoid function and its derivative. Beyond $|u| \approx 6$ the derivative is numerically zero: no gradient can flow back through a neuron in that state.

2.2 Forward propagation

For an input x , the network output is computed in two stages, each consisting of a matrix product followed by an element-wise nonlinearity:

$$U_{in} = w_1 x + w_{01} x_0, \quad U_{out} = f(U_{in}) \quad (\text{hidden layer}), \quad (2)$$

$$Y_{in} = w_2 U_{out} + w_{02} z_0, \quad y = f(Y_{in}) \quad (\text{output layer}), \quad (3)$$

where x_0 and z_0 are the bias inputs, usually equal to 1.

The role of the bias is often underestimated. Without it, the pre-activation of a neuron necessarily vanishes when all its inputs are zero, so its output is invariably $f(0) = 0.5$. The bias is the parameter that shifts the threshold of the activation function along the u axis; disabling it amounts to forcing every decision boundary to pass through the origin. Example 2 of Section 4 will illustrate the cost of that constraint concretely.

2.3 Gradient backpropagation

Training consists in minimising the quadratic error

$$E = \frac{1}{2} \sum_{j=1}^S (y_{d,j} - y_j)^2 \quad (4)$$

between the desired output y_d and the computed output y .

The central problem is to assign to each weight its contribution to the overall error. Backpropagation solves it by systematic application of the chain rule, in three steps:

1. propagation of the input through to the output;
2. evaluation of the output error, $e = y_d - y$;
3. backpropagation of that error towards the inputs and update of the weights.

The local gradients, traditionally denoted δ , are

$$\delta_2 = e \odot y \odot (1 - y) \quad (\text{output layer}), \quad (5)$$

$$\delta_1 = (w_2^\top \delta_2) \odot U_{out} \odot (1 - U_{out}) \quad (\text{hidden layer}), \quad (6)$$

where $\odot$ denotes the element-wise product. The weights are then corrected in proportion to the learning rate α :

$$w_2 \leftarrow w_2 + \alpha \delta_2 U_{out}^\top, \quad w_{02} \leftarrow w_{02} + \alpha \delta_2 z_0, \quad (7)$$

$$w_1 \leftarrow w_1 + \alpha \delta_1 x^\top, \quad w_{01} \leftarrow w_{01} + \alpha \delta_1 x_0. \quad (8)$$

Equation (6) contains the whole logic of the algorithm and deserves a pause. The term $w_2^\top \delta_2$ redistributes the output error towards the hidden neurons in proportion to their influence, which is backpropagation proper. The factor $U_{out} \odot (1 - U_{out})$ is the sigmoid derivative of each hidden neuron: it weights the received error by the local sensitivity of the neuron. It is this second factor that becomes zero under saturation, with the consequence analysed in Section 6.3.

2.4 Stopping criterion

Training is iterated as long as the maximum output error exceeds a fixed tolerance:

$$\max_j |y_j - y_{d,j}| > \text{tol}. \quad (9)$$

Choosing the infinity norm rather than the mean squared error is deliberate: it guarantees that *every* output satisfies the tolerance, whereas an average could hide a very poor output compensated by a very good one.

3 Program architecture

3.1 File organisation

The program is split into specialised scripts, all called from the main program.

Each data file defines the network *entirely*: dimensions N , K and S , initial weights, biases, learning rate, tolerance and desired output. The core of the program is therefore independent

Table 2. Organisation of the program.

File	Role
pmcmain.m	main program, entry point
dataselect.m	data selection menu and setting of the mode flags
data1.m, data2.m, data3.m	definition of examples 1 to 3
datass.m	training set of three observations
datasymbol.m	observation-by-observation training on the set
pmc.m	training loop and stopping criterion
propagation.m / retropropagation.m	explicit-loop version
propagation2.m / retropropagation2.m	vectorised version, biases trained
affichage.m	display of the results

of the data, as the assignment required. The dimensions themselves are never hard-coded: they are deduced from the sizes of the vectors supplied, through $N = \text{length}(x)$, $K = \text{length}(w01)$ and $S = \text{length}(y_d)$.

3.2 Operation

The main program `pmcmain.m` calls `dataselect.m`, which displays a menu. Depending on the choice, it loads the corresponding example and sets two flags: `CONDITIONS`, which controls exit from the program, and `CONDITIONS2`, which selects the data-set mode.

For a simple example the program runs the training loop `pmc.m` directly. For the training set, `datasymbol.m` presents the observations one at a time to the network, retaining the learned weights from one observation to the next, and calls `pmc.m` for each of them.

The training loop is the core of the program.

```

1  e = 1000;      % initial error, deliberately large
2  niter = 0;     % iteration counter
3
4  while max(abs(e)) > tol
5
6      if CONDITIONS2 == 1
7          propagation2;
8          e = yd - y;      % output error
9          retropropagation2;
10     else
11         propagation;
12         e = yd - y;      % output error
13         retropropagation;
14     end
15
16     niter = niter + 1;
17
18 end

```

Listing 1. `pmc.m`: training loop and stopping criterion.

At each iteration a forward pass computes the output y , the error $e = y_d - y$ is evaluated, and backpropagation corrects the weights. The loop stops as soon as $\max |e| \leq \text{tol}$. The initialisation $e = 1000$ guarantees that at least one iteration is executed, MATLAB having no *do-while*

construct.

3.3 The two implementations

Two implementations coexist, and comparing them is part of the purpose of the work.

Loop version. The files `propagation.m` and `retropropagation.m` follow the course equations step by step, neuron by neuron, with `for` loops nested over S , K and N . This version is used for examples 1 to 3 and serves as a pedagogical reference: every line of code corresponds exactly to one equation of Section 2, which makes it verifiable line by line.

```

1 % hidden layer
2 Uin = w1*x + w01*x0;      % hidden-neuron inputs
3 Uout = f(Uin);            % hidden-neuron outputs
4
5 % output layer
6 Yin = w2*Uout + w02*z0;
7 Yout = f(Yin);
8 y = Yout;                % network output

```

Listing 2. `propagation.m`: forward pass, a direct transcription of Equations (2) and (3).

Vectorised version. The files `propagation2.m` and `retropropagation2.m` replace all loops by matrix products, and additionally train the bias weights w_{01} and w_{02} . This is the most efficient form in MATLAB, whose interpreter is optimised for linear algebra.

```

1 w2old = w2;      % weights before the update
2
3 % output layer
4 delta2 = e .* y .* (1 - y);
5 w2 = w2 + alpha * delta2 * y1';
6 w02 = w02 + alpha * delta2 * z0;
7
8 % hidden layer
9 delta1 = (w2old' * delta2) .* y1 .* (1 - y1);
10 w1 = w1 + alpha * delta1 * x';
11 w01 = w01 + alpha * delta1 * x0;

```

Listing 3. `retropropagation2.m`: vectorised update, corresponding to Equations (5) to (8).

One detail of Listing 3 is essential and easy to miss: the computation of δ_1 uses `w2old`, that is, the output-layer weights *before* their update. This follows the theory, since the gradient is evaluated at a single point of parameter space: both layers must be corrected from the same network state. Using the already updated weights would introduce an inconsistency that does not make training diverge but slows it down and makes it dependent on the order in which the lines are written, a fault that is difficult to detect.

4 Test data

Four data sets were used; their characteristics are summarised in Table 3.

The desired outputs are $y_d = 1$ for example 1, $y_d = (0.5, 1)$ for example 2 and $y_d = (0.01, 0.99)$ for example 3. Example 4 is a small binary classification set: three observations with two raw input variables, for instance $x = (227, 138)$, and a label of 0 or 1.

Table 3. Characteristics of the four data sets.

Example	File	$N \times K \times S$	α	tol	Notable feature
1	data1	$2 \times 2 \times 1$	1	0.01	biases active, $x_0 = z_0 = 1$
2	data2	$2 \times 2 \times 2$	0.5	0.001	biases disabled , w_1 symmetric
3	data3	$2 \times 2 \times 2$	1.5	0.001	biases active, high α
4	datass	$2 \times 2 \times 1$, 3 obs.	1	0.005	raw inputs ≈ 230

Two features deserve to be flagged at once, since they govern most of the results.

Example 2 is configured with $x_0 = z_0 = 0$, hence with no effective bias, and with an initial weight matrix $w_1 = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}$ whose two rows are identical. Its target additionally contains a component at 0.5.

Example 4 uses unnormalised inputs of the order of 230, while the initial weights are of order unity. The product $w_1 x$ therefore immediately reaches several tens.

5 Results

Measurements were obtained under MATLAB R2024b. Execution times are measured with `tic/toc` over the complete training loop. The convergence curves of Figures 3 and 4 were obtained by instrumenting the loop to record $\max_j |e_j|$ at each iteration.

5.1 Examples 1 to 3

Table 4. Convergence of examples 1 to 3, loop version.

Example	Final y	y_d	Iterations	Time
1	0.990	1	3 709	0.37 s
2	(0.500, 0.999)	(0.5, 1)	528 537	36.2 s
3	(0.011, 0.989)	(0.01, 0.99)	5 835	0.39 s

All three examples do converge, that is, the stopping criterion (9) is eventually satisfied in every case. The final weights obtained are, for example 1,

$$w_1 = \begin{pmatrix} 0.700 & 0.763 \\ -0.200 & 1.155 \end{pmatrix}, \quad w_2 = \begin{pmatrix} 3.065 & 3.291 \end{pmatrix},$$

and for example 3,

$$w_1 = \begin{pmatrix} 0.217 & 1.546 \\ 0.315 & 1.607 \end{pmatrix}, \quad w_2 = \begin{pmatrix} -2.905 & -2.939 \\ 2.188 & 2.271 \end{pmatrix}.$$

A useful observation on example 1: the first column of w_1 remained exactly at its initial value $(0.700, -0.200)^\top$. This is expected rather than faulty, since the first component of the input is $x_1 = 0$: by the update rule (8), the correction applied to $w_1(k, i)$ is proportional to x_i , hence zero for $i = 1$. A weight connected to an identically zero input cannot learn.

Figure 3 reveals very different shapes. Examples 1 and 3 decrease steadily from the first it-

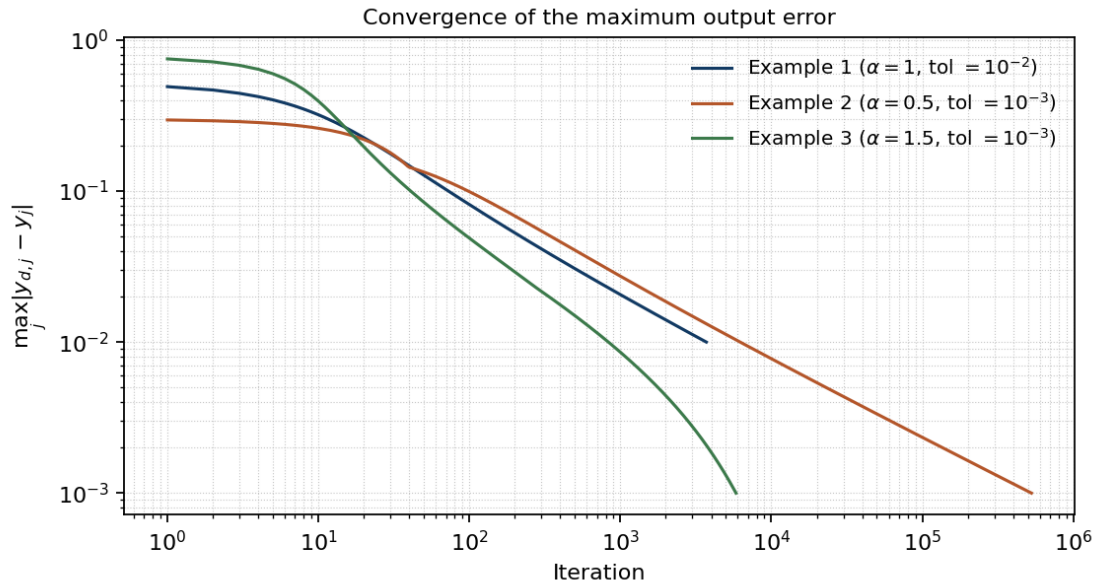


Figure 3. Convergence of the maximum output error for the three examples, logarithmic on both axes. Example 2 shows a long plateau before the error really begins to decrease.

erations, whereas example 2 stagnates for several hundred thousand iterations before crossing its tolerance. This difference is analysed in Section 6.2.

5.2 Training set

For the training set, handled with the vectorised version, each observation is trained to convergence with the weights retained from one observation to the next.

Table 5. Training set: iterations per observation. Total time **0.99 s**.

Observation	Input x	y_d	Iterations
1	(227, 138)	1	6 799
2	(229, 110)	0	6 875
3	(235, 124)	1	6 875

The final weights are

$$w_2 = \begin{pmatrix} 2.074 & 1.664 \end{pmatrix}, \quad w_{02} = 1.554,$$

while w_1 and w_{01} remain *exactly* at their initial values

$$w_1 = \begin{pmatrix} 0.75 & -0.30 \\ 0.02 & 0.13 \end{pmatrix}, \quad w_{01} = \begin{pmatrix} 0.2 \\ 0.1 \end{pmatrix}.$$

That w_1 and w_{01} do not move by a single significant digit, while the output layer trains normally, is not a defect of the program but a genuine phenomenon of the algorithm, analysed in Section 6.3.

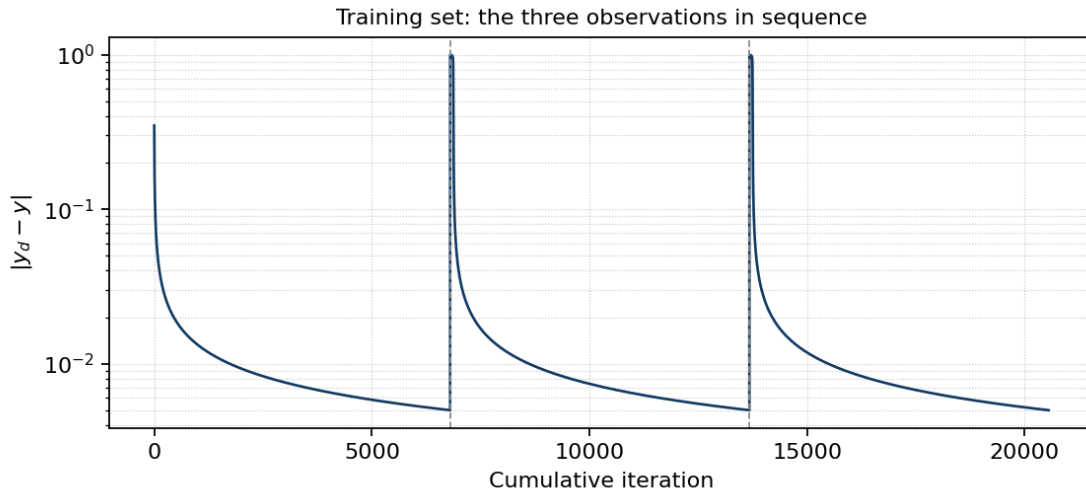


Figure 4. Training set: error during the three chained training phases. The vertical lines mark the transition from one observation to the next.

6 Discussion

6.1 Convergence speed of examples 1 and 3

Examples 1 and 3 converge in a few thousand iterations and in less than 0.4 s. Example 3, despite a tolerance ten times stricter than that of example 1 (10^{-3} against 10^{-2}) and a more demanding target, converges in a comparable time thanks to its higher learning rate ($\alpha = 1.5$ against $\alpha = 1$).

This is the expected behaviour of a fixed-step gradient descent: the step length is proportional to α , so a larger α speeds up the descent, as long as it does not cause oscillations about the minimum. There is a critical value beyond which the step systematically overshoots the minimum and training diverges; neither of these examples comes close to it, which is why the gain is purely beneficial here.

6.2 The pathological case of example 2

Example 2 requires 528 537 iterations and 36 s, a factor of roughly one hundred relative to the other two. Three causes combine, and it is instructive to separate them.

Disabled biases. With $x_0 = z_0 = 0$, the terms $w_{01}x_0$ and $w_{02}z_0$ of Equations (2) and (3) disappear. The network loses one degree of freedom per neuron, the one that adjusts the threshold of its activation function. Every decision boundary is forced through the origin, which is not always compatible with the target.

Symmetric initialisation. The two rows of w_1 are identical, so the two hidden neurons receive the same pre-activation and produce the same output. By (6) they then receive the same gradient and undergo the same correction: they remain identical at every iteration. The symmetry is never broken, and the network behaves as if it had only one hidden neuron, with the corresponding representational capacity. The final weights confirm the diagnosis, both rows of w_1 remaining equal to (1.066, 1.631).

Target at 0.5. The desired output contains a component $y_{d,1} = 0.5$. But the sigmoid equals exactly 0.5 only for a strictly zero pre-activation. The network must therefore drive the

activation of the first output neuron to zero *asymptotically*, which is intrinsically slow: the closer it gets, the smaller the gradient becomes. The final weights show that the network achieves this by nulling the first row of w_2 , which settles at $(0, 0)$.

This example, presumably designed for the purpose, illustrates the practical importance of three rules that are now systematic in machine learning: initialise the weights randomly and asymmetrically, keep the biases active, and express targets within a range that the activation function actually attains.

6.3 Sigmoid saturation on the training set

On the training set the hidden-layer weights w_1 and w_{01} do not move at all. The explanation lies entirely in the factor $U_{out} \odot (1 - U_{out})$ of Equation (6).

The inputs are raw and unnormalised, of the order of $x \approx 230$. With initial weights of order unity, the pre-activation of the hidden neurons is, for the third observation for instance,

$$U_{in} = w_1 x + w_{01} x_0 = \begin{pmatrix} 0.75 & -0.30 \\ 0.02 & 0.13 \end{pmatrix} \begin{pmatrix} 235 \\ 124 \end{pmatrix} + \begin{pmatrix} 0.2 \\ 0.1 \end{pmatrix} = \begin{pmatrix} 139.25 \\ 20.92 \end{pmatrix}. \quad (10)$$

At those values the sigmoid equals 1 to machine precision and its derivative $U_{out}(1 - U_{out})$ is numerically zero: $f'(139) \approx 10^{-61}$ and $f'(20.9) \approx 8 \times 10^{-10}$. By (6) the local gradient δ_1 is therefore zero, and by (8) the weights w_1 and w_{01} receive no correction at all. The gradient no longer flows through the hidden layer, which Figure 2 shows graphically.

All the learning is then carried by the output layer alone, w_2 and w_{02} , which suffices here to converge each observation individually, since both hidden outputs are saturated at 1 and the network effectively reduces to a single neuron with two constant inputs and a bias.

This is a classical limitation of sigmoid networks, known as the vanishing gradient problem, and the remedy is equally classical: the inputs must be **normalised**, for example rescaled to $[0, 1]$ or standardised, so that the pre-activations remain in the region where the sigmoid derivative is significant. With the data of this example, a simple division by 255 would bring U_{in} into a usable range and make the hidden layer genuinely trainable.

6.4 Value of the vectorised version

The vectorised version replaces three nested for loops by four matrix products per iteration. In MATLAB, whose interpreter is optimised for linear algebra and delegates matrix products to compiled libraries, this is both the fastest and the most readable form.

It also has a direct advantage with respect to the assignment: it handles networks of arbitrary dimensions without changing a single line of code, since the dimensions appear nowhere explicitly, whereas the loop version must know S , K and N in order to write its bounds. That is exactly the generality requirement. The loop version nevertheless retains its pedagogical value: it makes visible what the matrix product condenses, and it is the one that allows the equations to be checked one at a time.

7 Conclusion

This laboratory work produced a complete multilayer perceptron in MATLAB, from forward propagation to gradient backpropagation, in a general matrix form: the same program solves

the four data sets without any modification of its computational core, which meets the requirement of the assignment.

Beyond the programming, the analysis of the results brought out several fundamental phenomena of neural network training, which the data sets provided appear to have been constructed to reveal.

The influence of the learning rate on convergence speed appears when comparing examples 1 and 3: a higher α compensates for a stricter tolerance. The blockage of example 2 shows the combined effect of a symmetric initialisation, which permanently prevents the two hidden neurons from differentiating, of the absence of biases, which forces decision boundaries through the origin, and of a target that the sigmoid reaches only asymptotically. The training set finally illustrates sigmoid saturation in the presence of unnormalised inputs: computing the pre-activations explicitly, of the order of 10^2 , shows that the derivative of the activation falls below 10^{-9} and that the hidden layer simply stops learning.

These three observations converge on the good practices used systematically today: random weight initialisation to break symmetry, retention of the biases, normalisation of the input data and careful choice of the learning rate. Having met them as concrete difficulties, on a program written entirely by hand, gives them a content that a theoretical exposition transmits only with difficulty.

A natural extension of this work would be to add input normalisation and random initialisation, then measure the iteration counts again on the four data sets: example 2 would be expected to return to a cost comparable to the others, and the hidden layer of example 4 to participate in the learning at last.

References

- [1] D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River: Pearson, 2009.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [4] Y. LeCun, L. Bottou, G. B. Orr and K.-R. Müller, “Efficient BackProp,” in *Neural Networks: Tricks of the Trade*. Berlin: Springer, 1998, pp. 9–50.
- [5] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge: MIT Press, 2016.
- [6] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” *Proc. AISTATS*, pp. 249–256, 2010.

A Complete data sets

Table 6. Initial values of the four data sets.

	Example 1	Example 2	Example 3	Example 4
x	(0, 0.7)	(0.5, 1)	(0.05, 1)	3 obs., see Tab. 5
x_0	1	0	1	1
z_0	1	0	1	1
w_1	$\begin{pmatrix} 0.7 & -0.4 \\ -0.2 & 0.3 \end{pmatrix}$	$\begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}$	$\begin{pmatrix} 0.15 & 0.20 \\ 0.25 & 0.30 \end{pmatrix}$	$\begin{pmatrix} 0.75 & -0.30 \\ 0.02 & 0.13 \end{pmatrix}$
w_{01}	(0.4, 0.7)	(0, 0)	(0.35, 0.35)	(0.2, 0.1)
w_2	(0.5 0.1)	$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$	$\begin{pmatrix} 0.40 & 0.45 \\ 0.50 & 0.55 \end{pmatrix}$	(0.52 0.11)
w_{02}	-0.3	(0, 0)	(0.60, 0.60)	0
y_d	1	(0.5, 1)	(0.01, 0.99)	{1, 0, 1}
α	1	0.5	1.5	1
tol	0.01	0.001	0.001	0.005

B Reproducing the results

The code is available under the MIT licence at <https://github.com/KhalilEllahLAGHA/mlp-backpropagation-matlab>. To reproduce the results of Section 5, run `pmcmain.m` under MATLAB R2024b and select one of the four data sets from the menu. The convergence curves of Figures 3 and 4 are obtained by recording $\max_j |e_j|$ at each pass through the loop of Listing 1.